ИНЖЕНЕРНЫЙ CASE STUDY

IRIS Security RAG

Локальные ответы с цитатами для SOC-аналитиков

АВТОР	Max Nikiforov
ДАТА	6 августа 2026
СРЕДА	Windows 10 · CPU-only · 16 ГБ RAM
STACK	FastAPI · PostgreSQL · Qdrant · Ollama · Prometheus · Grafana

Production-style portfolio project: corpus engineering, hybrid retrieval, reranking, локальная генерация, evaluation и observability.

Краткое резюме

IRIS - локальная Retrieval-Augmented Generation система поддержки SOC-аналитиков. Она превращает разнородный корпус по информационной безопасности в ответы с цитатами и делает измеримыми компромиссы retrieval, generation и производительности.

Проект выходит за рамки минимального vector-search demo. В нём реализованы воспроизводимая загрузка источников, format-specific parsing, нормализация metadata, PostgreSQL и Qdrant, dense и lexical retrieval, weighted Reciprocal Rank Fusion, опциональный cross-encoder reranker, локальная генерация через Ollama, no-answer guard, FastAPI API, браузерный интерфейс, offline evaluation и observability на Prometheus/Grafana.

Лучшая retrieval-конфигурация достигла 100% Content Hit@5 и 0,972 Content MRR на 50 answerable-вопросах. Отдельный batch из 20 HTTP-запросов завершился без транспортных ошибок, а каждый принятый ответ содержал источники. Эксперимент также выявил два честных ограничения: генерация на CPU доминирует в общей latency, а текущий guard по retrieval score может отклонить релевантный контекст после reranking.

1. Постановка задачи

Знания по информационной безопасности распределены между разными форматами и уровнями абстракции:

- MITRE ATT&CK описывает техники атакующих;
- Sigma выражает detection logic в YAML;
- OWASP описывает риски безопасности приложений;
- runbooks задают локальные действия triage и response;
- материалы по risk management дают более общий контекст защитных мер.

Для ответа на один вопрос SOC-аналитику могут понадобиться сразу несколько источников. Keyword search теряет семантические совпадения, а чистый vector search может недостаточно учитывать точные индикаторы, technique IDs, команды и имена файлов. Сгенерированный ответ без проверяемых доказательств также неприемлем для операционной работы.

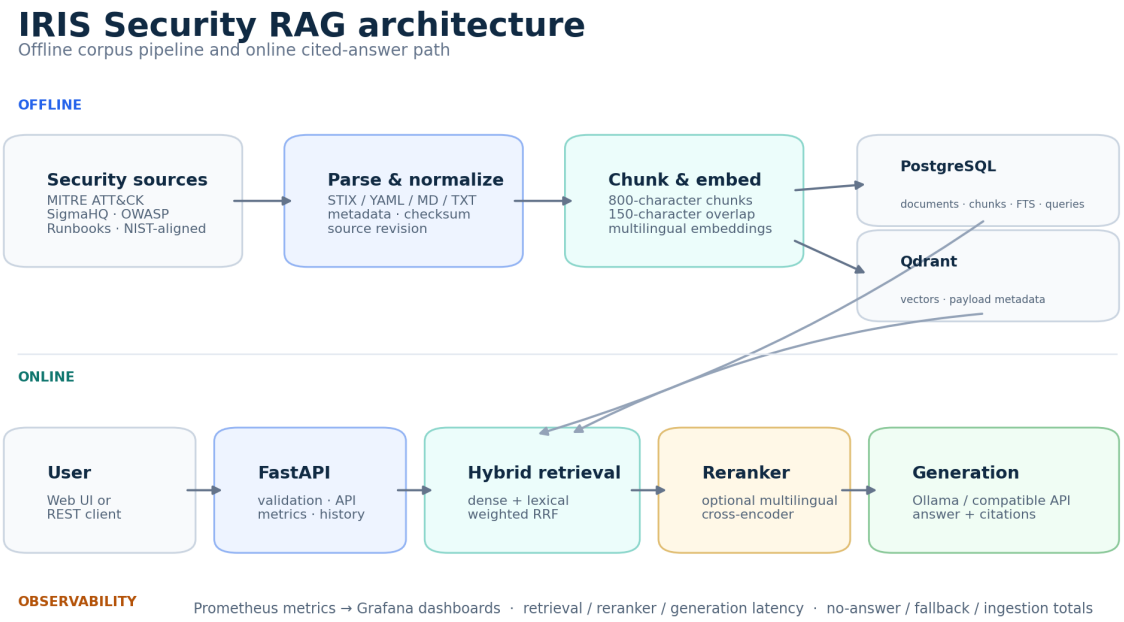
Поэтому IRIS решает четыре задачи: извлекает релевантный контекст, сохраняет provenance, отклоняет слабые результаты и показывает измеримые показатели качества и скорости.

2. Инженерные требования

Реализация строилась с учётом следующих ограничений:

1. **Local-first.** Основной эксперимент должен работать без платного cloud LLM.
2. **CPU-friendly stack.** Ноутбук разработчика имеет 16 ГБ RAM и не имеет дискретной GPU.
3. **Мультиязычные запросы.** Русские вопросы должны находить англоязычные security-документы.
4. **Цитаты по умолчанию.** Ответ обязан ссылаться на полученный контекст.
5. **Воспроизводимость.** Нужно сохранять ревизии официальных источников и evaluation artifacts.
6. **Честный no-answer.** Система должна отклонять недостаточный контекст, а не придумывать ответ.
7. **Наблюдаемость.** Retrieval, reranking, generation, fallback и ingestion должны быть измеримыми.

3. Архитектура системы



Архитектура IRIS

Offline data path

Corpus builder загружает выбранные MITRE ATT&CK STIX data, правила SigmaHQ и Markdown OWASP Top 10:2025 из официальных репозиториев. Он определяет commit репозитория, преобразует source-specific форматы в нормализованный Markdown и создаёт manifest с upstream URL, revision и SHA-256.

Ingestion pipeline извлекает metadata, выполняет детерминированный chunking и сохраняет документы и чанки в PostgreSQL. Отдельный indexing-шаг создаёт embeddings через `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` и загружает vectors с metadata payload в Qdrant.

Online query path

API принимает вопрос, `top_k`, retrieval strategy и флаг reranker. Dense candidates поступают из Qdrant, lexical candidates - из PostgreSQL full-text search. Hybrid mode объединяет rankings через weighted RRF. Опциональный multilingual cross-encoder переранжирует расширенный candidate pool. Лучшие контексты передаются generation service, который возвращает ответ с цитатами или контролируемый no-answer.

Prometheus инструментирует online path. Grafana показывает totals, query rate, retrieved chunks, fallback, no-answer, а также p50/p95 latency для retrieval, generation и полного запроса.

4. Корпус и происхождение данных

Источник	Документов	Чанков	Примечание
MITRE ATT&CK	30	277	Официальные STIX techniques, преобразованные в Markdown
Sigma	64	461	Detection rules, преобразованные из YAML
OWASP Top 10:2025	10	214	Официальный англоязычный Markdown
NIST-ориентированный ransomware summary	1	10	Составлен вручную, не является официальным файлом NIST
Русскоязычные SOC runbooks	3	32	Синтетические operational-документы
Всего	**108**	**994**	В среднем 9,2 чанка на документ

Manifest официальных источников содержит 100 сгенерированных документов: 30 MITRE, 60 Sigma и 10 OWASP. Ещё четыре legacy Sigma-документа, три runbook и один NIST-ориентированный обзор доводят индексированную базу до 108 документов.

Это разделение важно: репозиторий не выдаёт самостоятельно написанный материал за официальную документацию NIST или поставщика. Текущий audit не обнаружил пустых документов, legacy `source_type` или документов без чанков, но отметил неполные provenance-поля у восьми добавленных вручную или legacy-документов.

5. Retrieval design

Dense retrieval

Dense retrieval создаёт embedding мультиязычного вопроса и выполняет cosine search в Qdrant. Он быстр и понимает перефразирование, но точные security tokens могут теряться среди семантически похожих результатов.

Lexical retrieval

PostgreSQL full-text search сохраняет точные сигналы: ATT&CK IDs, command-line switches, process names и термины Sigma. Его слабая сторона - различие словаря между русскими вопросами и английскими документами.

Hybrid fusion

Hybrid mode объединяет dense и lexical rankings с помощью weighted Reciprocal Rank Fusion. RRF устойчив, потому что использует позиции результатов, а не пытается напрямую сравнить vector и lexical scores разных шкал.

Опциональный reranking

`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1` совместно оценивает вопрос и каждый candidate text. Это улучшает content-level ordering, но значительно дороже на CPU. Поэтому reranker доступен как опция, а не обязательный этап.

6. Генерация и защитное поведение

Generator получает только retrieved context и явные citation IDs. Локальная модель основного эксперимента - `qwen3:4b` через Ollama. Архитектура также поддерживает OpenAI-compatible endpoint и extractive provider.

Контракт ответа содержит:

- `answer` и `is_answerable`;
- confidence и rejection reason;
- источники с citation metadata;
- retrieval strategy и состояние reranker;
- total, retrieval, reranker и generation latency;
- provider и fallback metadata.

Retrieval-based guard запрещает генерацию ниже минимального score. Это снижает риск unsupported answer, но требует аккуратной калибровки, потому что fused retrieval score и cross-encoder score имеют разный смысл.

7. Методика evaluation

Golden dataset содержит 55 вопросов: 50 answerable и 5 unanswerable. Темы включают phishing, PowerShell, ransomware, MITRE techniques, Sigma detections, OWASP risks и negative cases.

Retrieval-конфигурации сравнивались в одинаковых условиях:

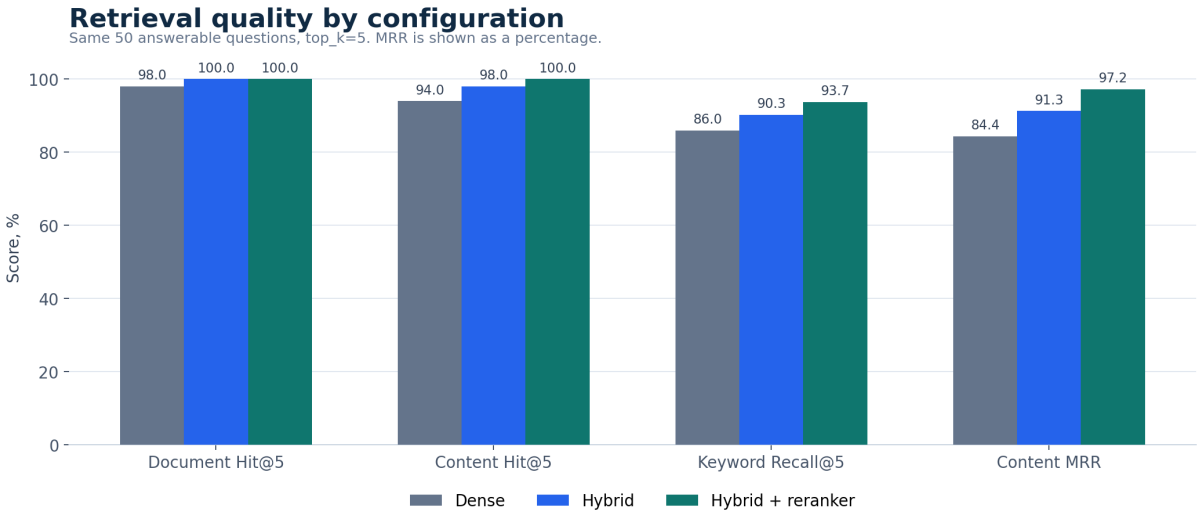
- один корпус и embeddings;
- одни и те же 50 answerable-вопросов;
- `top_k=5`;
- document-level и content-level разметка;
- keyword coverage ожидаемых понятий;
- per-query latency;
- без generator в retrieval comparison.

Основные метрики:

- **Document Hit@5:** есть ли релевантный документ в первой пятёрке;
- **Content Hit@5:** есть ли чанк с ожидаемым доказательством;
- **Keyword Recall@5:** покрытие понятий, ожидаемых в ответе;
- **MRR:** насколько высоко расположен первый релевантный результат.

8. Результаты retrieval

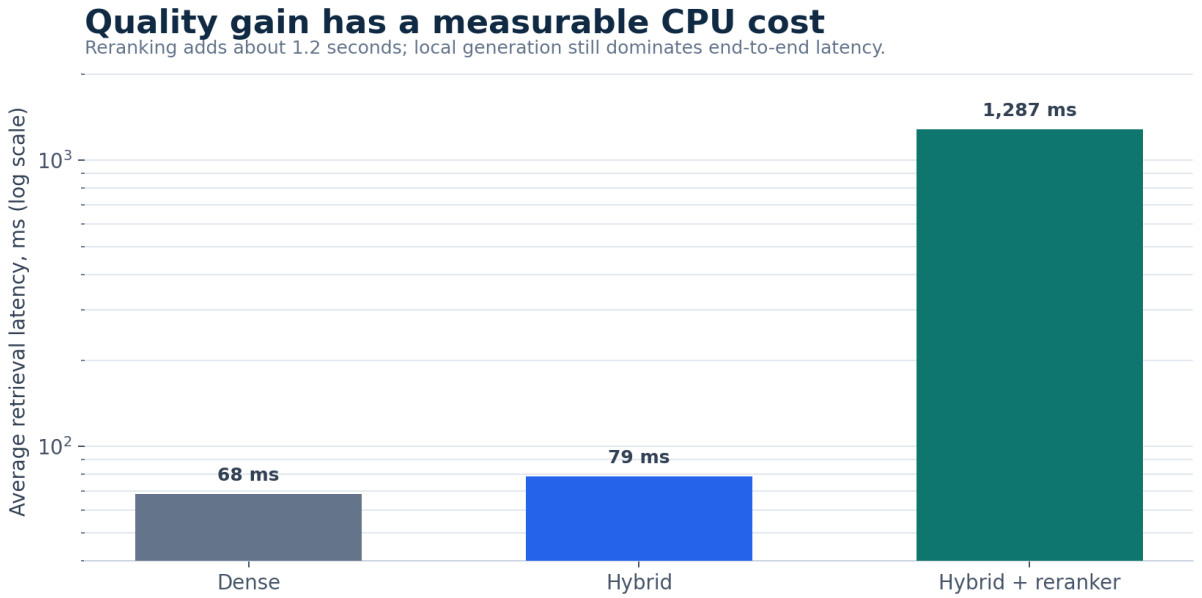
Конфигурация	Document Hit@5	Content Hit@5	Keyword Recall@5	Primary MRR	Content MRR	Avg latency
Dense	98%	94%	86,0%	0,940	0,844	68 мс
Hybrid	100%	98%	90,3%	0,990	0,913	79 мс
Hybrid + reranker	100%	100%	93,7%	0,967	0,972	1 287 мс



Сравнение retrieval quality

Hybrid - наиболее практичный default: он заметно повышает качество примерно за 10 мс дополнительной средней latency. Reranker даёт наиболее сильный content evidence и 100% Content Hit@5, но добавляет около 1,2 секунды на CPU.

Результаты также показывают, почему одной метрики недостаточно. Reranking повышает Content MRR с 0,913 до 0,972, но Primary/Document MRR снижается с 0,990 до 0,967. Одна headline-цифра скрыла бы этот компромисс.



Средняя retrieval latency конфигураций на CPU-only ноутбуке

9. End-to-end operational experiment

Retrieval benchmark не проверяет HTTP handling, generation, fallback и monitoring. Поэтому отдельный batch runner отправляет реальные запросы через `/api/v1/query`.

Конфигурация:

- один warm-up запрос;
- 20 измеряемых вопросов;

- последовательное выполнение;
- hybrid retrieval и reranker включены;
- `top_k=5`;
- timeout 300 секунд на запрос;
- локальная генерация `qwen3:4b` на CPU.

Метрика	Результат
Успешные HTTP-запросы	20 / 20
HTTP errors	0
Ответы	19
No-answer	1
Принятые ответы с источниками	100%
Fallback count	0
Средняя latency	63,45 с
p50 latency	70,68 с
p95 latency	83,44 с
Средняя latency reranker	1,18 с
Wall time	24 мин 55 с



Operational overview в Grafana

Grafana зарегистрировала 21 запрос, потому что warm-up также обновил Prometheus counters. Разделение latency показывает, что generation занимает десятки секунд, reranking - около секунды, а retrieval значительно меньше. Значит, основной local bottleneck - model inference, а не search.

10. Разбор ошибки: q004

Единственный no-answer получен на вопрос: «Какие вложения в письме считаются особенно опасными?». Первым результатом после reranking оказался правильный чанк phishing runbook с сильным cross-encoder score 5,70. Несмотря на это, guard запретил generation, потому что fused retrieval score 0,358 оказался ниже threshold 0,50.

Это не означает, что ответа не было в корпусе. Это означает, что guard откалиброван по score, который не отражает relevance после reranking. Более корректный вариант - использовать откалиброванный reranker threshold, небольшой answerability classifier или комбинированное правило, проверенное одновременно на answerable и unanswerable вопросах.

11. Какие навыки показывает проект

- Проектирование RAG как полноценного pipeline, а не одного framework call.
- Работа с разнородными реальными форматами источников.
- Разделение metadata/lexical задач PostgreSQL и vector search Qdrant.
- Controlled comparison retrieval-конфигураций по нескольким метрикам.
- No-answer logic и citations как продуктовые требования.
- Instrumentation реального HTTP path и поиск performance bottleneck.
- Честное описание ограничений и отрицательных результатов.

12. Ограничения

- Результаты получены на небольшом domain-specific наборе и не должны обобщаться на любые корпуса.
- Качество текста generation отдельно не оценивалось в operational batch.
- Пяти unanswerable-вопросов недостаточно для надёжной калибровки guard.
- CPU-only inference доминирует в общей latency.
- Восемь добавленных вручную или legacy-документов требуют полных provenance metadata.
- NIST-ориентированный summary и runbooks написаны вручную.
- Redis есть в инфраструктуре, но не подключён к online query path.
- Это portfolio prototype, а не hardened multi-user production service.

13. Следующие шаги

1. Перестроить guard вокруг final reranker evidence и откалибровать его на большем negative set.
2. Заменить NIST-ориентированный summary на официальный NIST document с зафиксированной revision.
3. Заполнить missing provenance и проверять эти поля в ingestion tests.
4. Добавить answer-faithfulness и citation-correctness evaluation.
5. Подключить Redis с явной cache invalidation policy.
6. Добавить CI, API container, rate limiting и authentication.
7. Сравнить GPU и hosted inference с CPU baseline.

14. Воспроизводимость

Ключевая конфигурация:

Компонент	Значение
Python	3.11+
Embeddings	`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`
Vector database	Qdrant
Relational / lexical store	PostgreSQL 16
Fusion	Weighted RRF, k=60
Reranker	`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`
Generator	Ollama, `qwen3:4b`
API	FastAPI
Metrics	Prometheus + Grafana
Experiment hardware	Windows 10 laptop, CPU-only, 16 ГБ RAM

Повтор retrieval comparison:

```
python scripts\compare_retrieval.py
```

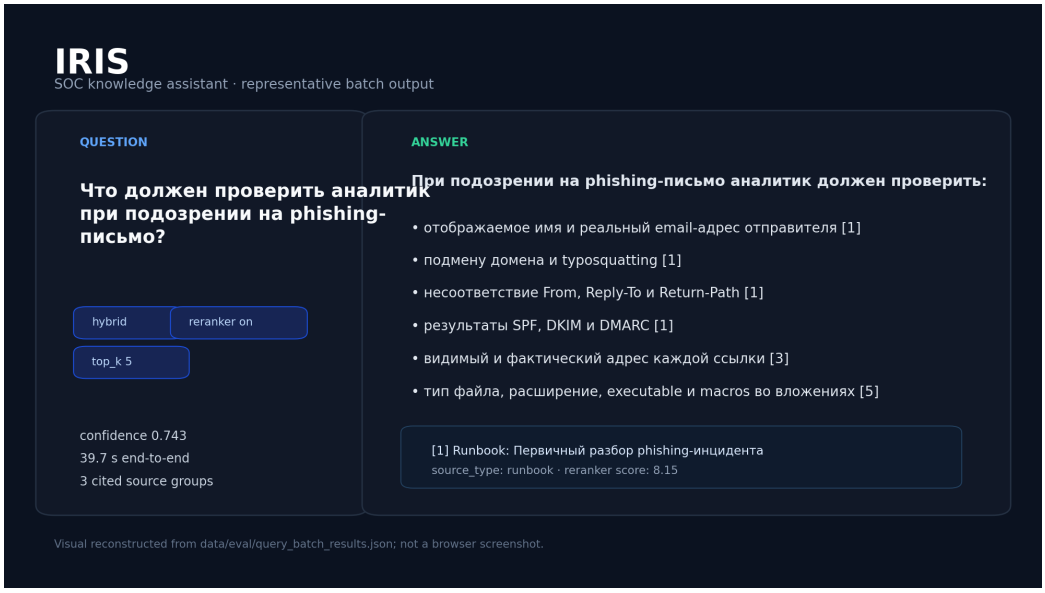
Повтор HTTP batch:

```
python scripts\run_query_batch.py --limit 20 --strategy hybrid --reranker
```

Raw artifacts находятся в `data/eval/`, а attribution открытых источников - в `THIRD_PARTY_NOTICES.md`.

15. Пример ответа

Изображение ниже реконструировано из первой успешной записи `data/eval/query_batch_results.json`. Оно сохраняет измеренный вопрос, основные утверждения ответа, citation IDs, confidence, latency и metadata главного источника, но не выдаётся за browser screenshot.



Пример ответа IRIS с цитатами, реконструированный из batch artifact

16. Evidence и ссылки на источники

Технический reviewer может проверить основные утверждения без полного воспроизведения окружения:

Артефакт	Что он подтверждает
`golden_questions.yml`	Набор вопросов и relevance expectations
`retrieval_comparison.json`	Per-query и aggregate retrieval metrics
`query_batch_results.json`	End-to-end HTTP outcomes и latency
`latest_corpus_audit.json`	Размер корпуса и metadata gaps
`corpus_sources.json`	Upstream revisions, URL и checksums

Основные внешние source projects:

- MITRE ATT&CK
- MITRE ATT&CK STIX data
- SigmaHQ rule repository
- OWASP Top 10 repository
- NIST ransomware publications